

Cited but Not Correct: Auditing Evidence-Grounded LLM Configuration Advice against Equally Informed Baselines

Anonymous ACL submission

Abstract

Language models are increasingly used to recommend system configurations from retrieved evidence, and their recommendations arrive with citations and natural-language explanations. Whether such advice adds anything beyond the supplied evidence, and whether the explanations state that evidence correctly, is rarely tested. We propose an audit protocol that gives an equally informed non-LLM baseline the same frozen evidence, ablates evidence sources, deletes cited versus matched uncited evidence to probe reliance, verifies every quantitative claim in an explanation against the evidence, and scores decisions on held-out outcomes labeled independently of any enforcement mechanism. We instantiate it with SemantOS, a testbed that separates evidence, LLM proposals and enforcement, on real Linux timer-slack and CPU-affinity interventions (8,000 measured events). Across ten LLMs, from 7B open-weight to hosted frontier models, evidence is necessary: without it, nine models mostly choose a worse configuration. It is also sufficient: with it, nine reproduce the table-lookup baseline’s choice and none improves on it. Explanations cite valid evidence yet misstate it: 16 of 28 audited quantitative claims from local models and 25 of 69 from hosted models are wrong, misattributed, overstated or in the wrong unit, and without evidence models invent meanings for opaque configuration identifiers. Deletion reveals citation-specific reliance for only one model. On the same testbed, a calibrated risk gate misses up to 20% of unsafe runs under a period shift. We release prompts, raw responses, annotations and measurement logs.

1 Introduction

Language models now act as configuration advisors: they read manuals and measurements, propose database or kernel settings, and justify each proposal in natural language (Trummer, 2022; Lao

et al., 2024; Giannakouris and Trummer, 2025; Chen et al., 2024; Lin et al., 2025). Such advisors are a form of evidence-grounded generation: a retrieval-augmented model (Lewis et al., 2020) produces a decision together with citations and an explanation. The NLP community has developed careful methods for this setting, including citation quality (Gao et al., 2023), attribution to identified sources (Rashkin et al., 2023), claim-level factual precision (Min et al., 2023) and explanation faithfulness (Jacovi and Goldberg, 2020; Turpin et al., 2023). Configuration advisors, by contrast, are usually judged end to end by the performance of the configuration they choose.

End-to-end performance leaves three questions open. *Added value*: does the model choose better than a conventional selector that receives the same evidence, or does it only repackage that evidence? *Explanation correctness*: when an explanation cites valid evidence, does it state that evidence correctly? *Reliance*: does the cited evidence actually influence the decision? A fourth question concerns acting on advice: whether a gate that screens proposals is judged by outcomes it does not itself define.

We answer these questions with an audit protocol and a testbed. The testbed, SemantOS, separates evidence (a typed interaction graph and retrieved measurement traces), proposals (an LLM that emits a complete configuration bundle with citations) and enforcement (a runtime that validates, vetoes or stages the bundle). Because the evidence is an explicit, frozen object, the same evidence can be handed to a non-LLM selector, ablated, or partially deleted; because enforcement is separate, its errors can be counted against labels computed from raw measurements.

Contributions. (i) An audit protocol for evidence-grounded LLM advice that combines a matched-information non-LLM baseline, evidence-source ablations, cited versus matched uncited

deletion, claim-level verification of explanations against the supplied evidence, and independent outcome labels, with training, retrieval, calibration and test roles frozen before test data are opened (§4). (ii) An audit of ten LLMs on real Linux interventions, reported with every denominator (§5). Evidence is necessary but, here, sufficient: models given the evidence match table lookup and never beat it. Explanations with valid citations frequently misreport ranges, extremes and units or overstate regularities, and deletion rarely shows citation-specific reliance. (iii) The SemantOS testbed with a bundle-level control contract, a class-conditional rank gate and a bounded actuator, together with released prompts, responses, annotations and logs (§3). The gate result is a cautionary example of selective prediction under shift.

The task is deliberately small, one workload family on one host, and its default configuration is best, so the audit can reveal failures of added value but not successes; we make no LLM-superiority, deployment-efficacy or production-safety claim (see Limitations).

2 Related Work

Grounded generation and attribution. Retrieval-augmented generation conditions outputs on retrieved documents (Lewis et al., 2020). ALCE evaluates whether generated statements are supported by the passages they cite (Gao et al., 2023), AIS defines attribution to identified sources (Rashkin et al., 2023), and FActScore decomposes long outputs into atomic facts checked against a knowledge source (Min et al., 2023). We apply claim-level verification to a structured, numeric evidence set, where support can be checked exactly against the prompt rather than against the open web, and we pair it with a decision-level baseline that receives the same evidence.

Faithfulness of explanations. Faithfulness must be evaluated separately from plausibility (Jacovi and Goldberg, 2020). Chain-of-thought explanations can omit the factors that drive a prediction (Turpin et al., 2023), and interventions on the reasoning test whether it matters (Lanham et al., 2023). ContextCite attributes generations to context by ablation (Cohen-Wang et al., 2024). We adopt matched deletion: removing cited evidence and, as a control, the same number of uncited items of the same kind.

LLMs as optimizers and configuration advisors. LLMs can optimize by proposing candidates from past evaluations (Yang et al., 2024). DB-BERT mines tuning hints from manuals (Trummer, 2022), GPTuner uses LLM-extracted knowledge to prune a Bayesian-optimization search (Lao et al., 2024), and λ -Tune generates complete configurations (Gianakouris and Trummer, 2025); AutoOS and TuneAgent tune Linux kernel build options with LLM agents (Chen et al., 2024; Lin et al., 2025). Classical tuners use Gaussian processes, reinforcement learning or black-box optimization (Van Aken et al., 2017; Zhang et al., 2019; Herodotou et al., 2020; Snoek et al., 2012; Golovin et al., 2017; Lee et al., 2022). These works report end-to-end gains; we instead ask what the language model adds over a selector given the same evidence, and whether its stated reasons are correct. We do not compare head to head with these systems.

Uncertainty and selective prediction. Self-consistency aggregates sampled outputs (Wang et al., 2023); LLMs can partly predict their own correctness (Kadavath et al., 2022), and semantic entropy measures uncertainty over meanings (Kuhn et al., 2023). Selective classification abstains to meet a target risk (Geifman and El-Yaniv, 2017), and conformal methods give distribution-free guarantees for language-model outputs (Quach et al., 2024) and monotone losses (Angelopoulos et al., 2024), with extensions for non-exchangeable or drifting data (Farinhas et al., 2024; Barber et al., 2023; Gibbs and Candès, 2021). None implies that a window reset after a detected change (Bifet and Gavaldà, 2007) restores a fixed risk level. Our rank gate is a restricted class-conditional instance of this family, and we report where its assumptions fail.

3 Testbed: SemantOS

SemantOS is a control loop with five components: telemetry, a graph and trace store, a reasoner, a runtime and an operator console (Figure 1). Its purpose here is to make the evidence, the proposal and the enforcement decision separately observable.

Evidence. The store holds a typed knob-interaction graph and measurement traces. For a lower-is-better loss L and baseline values a_0, b_0 , an interaction edge requires the contrast $I_{ab} = L(a, b) - L(a, b_0) - L(a_0, b) + L(a_0, b_0)$ from independently measured configurations, with a bootstrap interval over runs that excludes zero; an edge

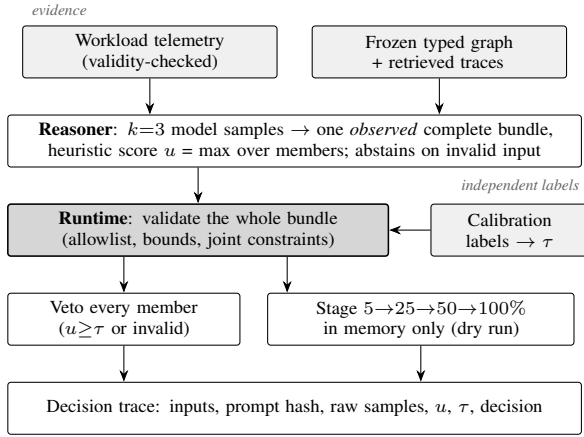


Figure 1: SemantOS decision path. Evidence, proposal and enforcement are separate components, so the same frozen evidence can be given to a non-LLM selector, ablated or partially deleted, and gate errors can be counted against independently defined labels. The REST runtime is a dry run; a separate per-thread actuator applies, verifies and restores timer slack and affinity in owned child processes.

records its scope, values, measurement IDs and interval, and describes an empirical interaction rather than a causal coupling (Appendix A).

Proposals. The reasoner retrieves typed neighbors and lexically matched trace text (Lewis et al., 2020) and requests $k = 3$ samples (Wang et al., 2023). It returns a modal *complete* bundle actually present in a response, rather than splicing per-knob winners into an unseen combination, together with the supplied context, prompt hash and raw samples. For a knob–value pair proposed in v samples, a heuristic score $u = 0.7(1 - v/k) + 0.3(1 - w)$, clipped to $[0.02, 1]$, combines disagreement with the strongest retrieved synergy weight w ; it is not a calibrated probability. Failed samples stay in the denominator, and the reasoner abstains when no model responds or the context budget would force truncation.

Enforcement. The runtime validates a whole bundle (allowlisted knobs, bounds, joint constraints, no duplicates) and treats it atomically: if any member score satisfies $u \geq \tau$, every member is vetoed; otherwise the bundle enters a staged rollout. The REST runtime is a dry run that stages bundles in memory and never writes kernel settings; the full contract is in Appendix A.

Rank gate. Let $Y = 1$ denote an unsafe outcome defined from raw observations, never from whether a veto or rollback occurred.

Proposition 1 (Restricted rank gate). *Fix a score function before calibration. Suppose n unsafe calibration scores and the next unsafe test score are jointly exchangeable. Set $j = \lfloor \alpha(n + 1) \rfloor$ for $0 < \alpha < 1$. If $j = 0$, veto all; otherwise let τ be the j th smallest unsafe calibration score and accept only when $u < \tau$. Then the unsafe-class acceptance probability, marginal over calibration and test scores, is at most $j/(n + 1) \leq \alpha$.*

Proof. With distinct scores the test score has a uniform rank among $n + 1$ exchangeable scores and lies below the j th calibration order statistic only in the first j ranks; ties with strict acceptance only reduce this probability. $\square$

The bound controls misses among unsafe actions, $\Pr(A = 1 \mid Y = 1)$, not the unsafe fraction among accepted actions; for bundles, u is the maximum member score and Y the bundle outcome. A cost-selected threshold is capped at the rank threshold, and no unsafe calibration example implies veto-all. An expected-cost identity for gated acceptance and a discussion of drift are in Appendix A.

Actuator. A separate adapter snapshots a child thread’s timer slack and CPU affinity (Linux man-pages project, 2026a,b), validates the target, applies both settings, verifies readback and restores both originals in a finally path, including after partial application. Updates are sequential, not an atomic kernel transaction.

4 Audit Protocol

Task. An advisor selects one of four per-thread configurations: timer slack of $50 \mu\text{s}$ or 1 ms (set by PR_SET_TIMERSLACK) crossed with affinity to all four CPUs or to CPU 0, named s50000_aall, s50000_aone, s1000000_aall and s1000000_aone. The objective is the 95th-percentile (P95) lateness of periodic wake-ups against absolute monotonic-clock targets, and s50000_aall is the explicit control. Every measurement runs in a fresh owned child process that checks kernel readback, on an Intel Core i5-3570 host (four logical CPUs, Linux 6.17); no host-wide setting is changed.

Role separation. The plan was saved before acquisition. Training, retrieval, calibration and test use wake-up periods of 2, 3, 4 and $\{5, 8\}$ ms. Each period has ten randomized blocks of the four configurations, with five warm-ups and 40 recorded events per child: 200 processes and 8,000 events.

Roles are period-disjoint but share one workload family and host. Selections and thresholds are frozen, with hashes, before calibration and test data are opened, and a validator rejects role overlap or unknown lineage.

Frozen evidence. The evidence given to every advisor contains three kinds of items: the training table (mean training-run P95 per configuration), an interaction-graph item (the training contrast, $-18.6 \mu\text{s}$, whose prespecified bootstrap interval includes zero, so the graph has no edge), and 40 retrieval items, each a single retrieval run’s configuration, period and P95. Test outcomes never enter a prompt.

Equally informed baselines. The table selector minimizes mean training P95; a saturated two-factor model of the same four means adds no information. Both use the same 40 training runs as the LLMs and no online trials. Unchanged-control and seeded uniform-random selectors are references. Matched-budget random search, ϵ -greedy Q-learning ($\epsilon = 0.1$), UCB1 (Auer et al., 2002) and Gaussian-process expected-improvement BO (Snoek et al., 2012) each evaluate $B \in \{4, 8, 16, 40\}$ training runs, starting with one per configuration and drawing recorded runs with replacement; $B = 40$ matches the table selector.

LLM conditions. Qwen2.5 7B and 14B (Qwen Team, 2024), Llama-3.1-8B (Grattafiori et al., 2024) and Phi-4 (Abdin et al., 2024) run locally as hashed 4-bit GGUF files in llama.cpp. Llama-3.3-70B, Qwen3-235B-A22B (Yang et al., 2025), DeepSeek-V3.2 (DeepSeek-AI, 2025), Nemotron-3-Ultra-550B (NVIDIA, 2026), Gemini 3.8 Flash (Google DeepMind, 2026b) and Gemini 3.1 Pro preview (Google DeepMind, 2026a) run through OpenRouter; their weights cannot be hashed. All models receive the same system prompt, JSON evidence and schema (a configuration from the allowed list, at most six citations restricted to supplied IDs, and an explanation of at most 400 characters; Appendix C), with temperature 0 and five seeds. Four input variants are compared: full evidence, no graph, no retrieval, and no evidence.

Reliance by matched deletion. For each seed, we delete the evidence items cited in the full-context answer and, separately, the same number of uncited items of the same kinds. A decision that

changes only when cited items are deleted indicates citation-specific reliance. A seed is non-estimable when no same-kind uncited control exists; in particular, the training table is the only table item, so an answer citing it has no matched control.

Claim verification. Every quantitative or comparative claim in each distinct explanation is checked against the supplied prompt evidence and labeled *correct*, *overstated*, *wrong* (including partly wrong), *wrong unit*, *wrong attribution* (a value assigned to the wrong evidence source) or, for no-evidence prompts, *unsupported prior*. A claim is *misstated* if it is wrong, misattributed, overstated or in the wrong unit. A statement that a configuration is “consistently” or “always” lowest is labeled overstated, since the chosen configuration is lowest in 8 of 10 retrieval blocks. Local models returned byte-identical text across seeds, so each distinct explanation is audited once in every variant; hosted models varied across seeds and providers, so all distinct full-context explanations are audited. The labels were assigned by one annotator (an author) and are released per claim.

Outcomes and scoring. Each fixed choice is scored by paired offline replay on the measured test grid: the chosen configuration’s run P95 in each block, and its paired change relative to the control in the same block, with a Student- t interval across the ten blocks. A test run is *unsafe* when more than 10% of its 40 events exceed a predeclared 200 μs deadline, a label computed from timing alone.

5 Results

5.1 Decisions: Evidence Is Necessary and Sufficient

Both informed non-LLM selectors choose the control, so their paired improvement is exactly zero; control test P95 is 179.7 ± 102.0 and $155.7 \pm 64.4 \mu\text{s}$ at 5 and 8 ms (95% half-widths across ten blocks; Appendix B). The control is also best on the test grid, so any deviation is a loss. The matched-budget optimizers converge on it too, choosing it in 98–99% of 1,000 seeds at $B = 40$; at small budgets, noisy single-CPU runs mislead them (Table 1).

Table 2 gives each model’s choice under every input condition. With evidence, nine of ten models choose the control in all three evidence variants, exactly as table lookup does; Llama-3.1-8B instead picks s50000_aone, which is worse on re-

Table 1: Matched-budget replay: % of 1,000 seeds choosing control/expected paired P95 change at 5 ms (%); last column: mean training evaluations of 1 ms settings at $B = 40$.

Method	$B = 4$	$B = 8$	$B = 16$	$B = 40$	1 ms evals
Random search	62/-55	74/-38	85/-22	98/-3	19.8
ϵ -greedy (RL)	65/-51	86/-20	95/-8	99/-1	3.8
UCB1 (RL)	65/-51	77/-33	91/-12	99/-1	12.0
GP-EI (BO)	65/-51	83/-25	95/-8	99/-1	2.5

Table 2: LLM decisions with evidence (full, no graph, no retrieval), without it, and after deleting cited or matched uncited evidence; superscripts count seeds; n/e: non-estimable; [†]hosted. Paired test P95 change versus control at 5 and 8 ms (%; 95% half-width): ctrl 0; 50/1 -145 ± 125 and -180 ± 156 ; 1m/all -599 ± 188 and -631 ± 179 ; 1m/1 -631 ± 183 and -699 ± 206 .

Model	Evidence	None	Del. cited	Del. unc.
Qwen-7B	ctrl	50/1	n/e	n/e
Llama-8B	50/1	50/1	1m/all	ctrl
Qwen-14B	ctrl	50/1	n/e	n/e
Phi-4	ctrl	1m/1	50/1	ctrl
Llama-70B [†]	ctrl	1m/1	ctrl	ctrl
Qwen3-235B [†]	ctrl	1m/1 ³	ctrl 50/1	ctrl
DeepSeek-V3.2 [†]	ctrl	50/1	n/e	n/e
Nemotron-550B [†]	ctrl	ctrl	ctrl	ctrl
Gemini-Flash [†]	ctrl	50/1	n/e	n/e
Gemini-Pro [†]	ctrl	50/1	n/e	n/e

play ($-145 \pm 125\%$ and $-180 \pm 156\%$). Without evidence, every model except Nemotron, which declares the default, chooses a worse setting in at least four of five seeds, usually single-CPU affinity with $50 \mu\text{s}$ slack (six models) or with 1 ms slack (three). The evidence is therefore necessary; in this task it is also sufficient, since no model improves on the selector that simply reads it. 249 of 250 calls returned schema-valid output.

5.2 Explanations: Cited but Not Correct

Every citation in every valid answer refers to a supplied item, since the schema permits nothing else. The explanations are nevertheless often wrong about what those items say (Table 3): 16 of 28 audited claims from local models and 25 of 69 from hosted models are misstated. Errors fall into four recurring patterns (Table 4).

Overstated regularities. Thirteen of the 19 overstatements assert that the chosen configuration is lowest “consistently” or in “all” retrieval runs, whereas it is lowest in 8 of 10 retrieval blocks; the other six claim “significant” differences that no test in the evidence supports.

Range and extremum errors. All 11 wrong

Table 3: Manual verification of quantitative and comparative claims in explanations against the supplied evidence. Corr.: correct; Over.: overstated; Wrong: wrong or partly wrong value; Unit/Src: wrong unit or evidence source; Prior: unsupported prior without evidence; Misst.: wrong, misattributed, overstated or wrong unit. Generated by `claim_taxonomy.py`.

Model	n	Corr.	Over.	Wrong	Unit/Src	Prior	Misst.
Qwen-7B	5	1	1	1	1	1	3
Llama-8B	7	—	—	6	—	1	5
Qwen-14B	6	1	1	—	3	1	4
Phi-4	10	5	3	1	—	1	4
Local, all variants	28	7	5	8	4	4	16
Llama-70B	7	5	—	2	—	—	2
Qwen3-235B	6	3	3	—	—	—	3
DeepSeek-V3.2	16	10	2	4	—	—	6
Nemotron-550B	23	13	5	5	—	—	10
Gemini-Flash	12	10	2	—	—	—	2
Gemini-Pro	5	3	2	—	—	—	2
Hosted, full context	69	44	14	11	—	—	25

Table 4: Examples of misstated claims (paraphrased as audited) and the check against the supplied evidence; all latencies in μs .

Model, label	Claim	Evidence
Gemini-Pro [†] , overstated	control “consistently the lowest in retrieval runs”	lowest in 8 of 10 blocks
Nemotron [†] , wrong	s50000_aone ranges from 85.0 to 430.7	maximum is 571.84
DeepSeek [†] , wrong	control “consistently lower than any” s50000_aone run	ranges overlap: 85.033 < 183.217
Qwen-7B, wrong source	training table gives control 126.36	126.36 is one retrieval run; training mean 132.94
Qwen-14B, wrong unit	training table gives control 132.9413 ms	evidence is in μs
Qwen-14B, prior	“50000” denotes smaller dataset size	a no evidence given; name is opaque

values from hosted models misreport a range, an extremum or a cross-configuration ordering. Nemotron repeatedly gives the s50000_aone range as 85.0–430.7 μs (its maximum is 571.84), and DeepSeek-V3.2 states that the control is “consistently lower than any” s50000_aone run although the ranges overlap ($85.033 < 183.217$). Llama-3.1-8B reports 571.84 μs , the *maximum* of s50000_aone, as the lowest P95, consistent with its wrong choice.

Source and unit confusion. Qwen2.5-7B attributes the retrieval value 126.36 to the training table, whose mean is 132.94; Qwen2.5-14B reports the training mean 132.9413 in milliseconds in three explanations, although the evidence declares microseconds.

Invented semantics. Without evidence, four local models justify their choice by assigning meanings to opaque identifiers, for instance reading “50000” as a smaller dataset size or “one” as a more aggressive approach. The configuration names carry no such information.

These errors are largely decoupled from the decision: 35 of the 41 misstated claims accompany a choice of the control. Valid citations thus establish provenance, and a correct decision can coexist with an incorrect account of the evidence that supports it.

5.3 Reliance: Rarely Estimable, Rarely Specific

Five models cite the training table in their full-context answers, so matched deletion is non-estimable for them. Among estimable models, only Phi-4 shows citation-specific reliance: deleting its six cited retrieval runs changes its choice in all five seeds while matched uncited deletion does not, but the new choice (s50000_aone) contradicts the training table that remains in the prompt. Llama-3.1-8B changes its choice under both deletions, which indicates sensitivity to context rather than reliance on what it cites. Llama-3.3-70B, Qwen3 and Nemotron change under neither, which is consistent with the remaining evidence still favoring the control but uninformative about reliance. Deletion tests therefore need evidence sets with matched, redundant items per kind to be informative.

5.4 Enforcement: A Calibrated Gate under Shift

For the gate, the score is a configuration’s training deadline-miss fraction, fixed before calibration, and calibration uses only 4 ms runs; all three pre-specified $\alpha \in \{.05, .10, .20\}$ yield $\tau = .9225$ because the unsafe-score order statistics are tied. Every candidate is executed in an isolated process, so accepted and vetoed candidates both have observed outcomes (Figure 2).

Unsafe acceptance is $5/25 = 20.0\%$ and $4/24 = 16.7\%$ among unsafe runs, above the nominal 5% and 10% levels; unsafe fractions among accepted runs are $5/20$ and $4/20$, and veto precision is $20/20$ in each period. The gate stays frozen throughout test. As with LLM explanations, a nominally calibrated signal is only as good as the exchangeability it assumes, and a period shift is enough to break it.

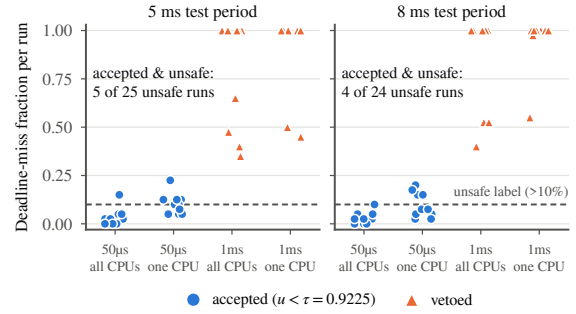


Figure 2: Frozen-gate replay on the 80 held-out test runs; each point is one run. Positive means unsafe (dashed line) and a gate-positive decision is veto. Counts at every tested α (all give $\tau = .9225$): 5 ms, 20 accepted / 20 vetoed, TP 20, FP 0, FN 5, TN 15; 8 ms, 20 / 20, TP 20, FP 0, FN 4, TN 16. Blue points above the line are the accepted unsafe runs. Generated from raw logs by `plot_gate_replay.py`.

Separately, the actuator executed all four configurations in ten blocks at each test period, observing stages of 8, 16 and 32 events and stopping when a stage’s deadline-miss fraction exceeded 10%. Of 80 executions, 66 stopped on a breach and 14 completed; all 80 restored and read back both original settings.

6 Discussion

What the protocol shows. Each component isolates a different failure. The matched-information baseline separates what the evidence determines from what the model adds; here the answer is nothing, a result invisible to an evaluation that compares an LLM advisor only with the unchanged default or with a cold-start optimizer. Evidence ablation shows the models do read the evidence, since removing it degrades nine of ten. Claim verification shows that reading is not the same as reporting: explanations with valid citations misstate a third or more of their quantitative content. Deletion shows that citation lists are weak evidence of reliance, and that reliance can only be estimated when the evidence contains matched alternatives.

Recommendations. For evaluating evidence-grounded advisors we recommend (i) giving a non-LLM selector exactly the model’s evidence and reporting both; (ii) verifying explanation claims against the supplied evidence, with special attention to quantifiers (“consistently”, “all”), ranges and units, which account for most misstatements at every model size; (iii) designing evidence sets so

that deletion controls exist; and (iv) labeling outcomes from raw observations rather than from the enforcement mechanism under test.

When could an LLM add value? Our task gives the evidence everything needed to decide. An LLM could add value when the evidence is incomplete and prior knowledge, for example from documentation, carries information the measurements lack. Testing this requires tasks where the default is not optimal and where the protocol’s baseline receives the same documents, which we leave to future work.

7 Conclusion

We audited evidence-grounded LLM configuration advice with a protocol that holds the evidence fixed across LLM and non-LLM advisors, ablates and deletes it, verifies explanation claims against it, and scores decisions on independently labeled held-out outcomes. On real Linux interventions, ten LLMs needed the evidence to choose well but added nothing beyond a selector that reads it, and their explanations frequently misstated the evidence they validly cited. A calibrated gate on the same testbed missed more unsafe runs than its nominal level under a period shift. Citation validity, nominal calibration and a correct final choice are each weaker evidence than they appear; the protocol and released data make these gaps measurable.

Limitations

Scope of the prototype. The REST runtime is a dry run: it never writes kernel settings, and the separate actuator changes only an owned child thread’s timer slack and affinity. The artifact therefore demonstrates a decision contract, a bounded per-thread actuator, raw measurements and non-LLM selector replay, but not a deployed VM-sysctl bundle actuator, traffic isolation or a verified trained checkpoint. We make no deployment-efficacy, production-safety or LLM-superiority claim; Section 4 and Appendix A list the matched comparisons and deployment requirements such claims require. The staged adapter is not wired into the REST VM-sysctl runtime and does not establish traffic isolation or recovery from irreversible effects. The reasoner awaits model samples synchronously, so we make no fast-path latency or resource-footprint claim; a nonblocking deployment would separate synchronous enforcement from asynchronous proposals and version

the objective weights, policy, graph and calibration snapshots per decision. Logs are append-only files within a process, without tamper resistance. Prompting a model to obey graph edges does not enforce those edges; independent validation is required, and the current REST runtime enforces only its explicit example constraints.

Task and annotation. The audited task has four candidate configurations, its evidence determines the best choice, and the default is best on the test grid; the audit can therefore expose missing added value and misreported evidence, but it cannot show that an LLM adds value on harder tasks. Claim labels were assigned by a single annotator (an author) without an agreement study; the per-claim labels and checks are released so that they can be re-annotated. The claim sample is small (97 claims), local explanations were byte-identical across seeds, and five seeds under greedy decoding are not independent samples.

Evaluation breadth. All controlled measurements use one periodic workload family on one shared host, with roles separated by period rather than by workload family or hardware. Measured graph induction on the training sweep emits no edge. Selector, BO and RL comparisons replay recorded runs rather than executing closed loops, and control is best on the tested grid, so these comparisons can reveal only losses. Run-level intervals describe variability on a shared host and do not account for uncontrolled interference, warm-cache bias or correlated system state; exploratory intervals are unadjusted for multiple comparisons and temporal dependence, and the closed-loop local load does not characterize queueing under an externally fixed arrival rate. The model audit is retrospective and small, and hosted runs are not reproducible bit for bit. The available telemetry prototype contains scheduler and block-I/O demonstration proxies, including timestamp-derived buckets; those are not end-to-end request latencies and must not calibrate an empirical SLO claim. The local benchmarks record completion, failure and deadline data for their narrow tasks but do not substitute for web serving, TPC-C, Kafka/Spark, GPU inference or audio pipelines.

Risk statements. The rank gate’s assumptions can fail under drift, selective labeling, or a score model updated using calibration data, and the observed miss rates under a period shift already ex-

ceed the nominal level. Because calibration and test periods differ, runs share a host, and the proposition is marginal under exchangeability, the gate replay neither verifies its deployment assumptions nor establishes recovery. A canary can suffer harm before detection, and rollback cannot undo every side effect. No no-worse-than-default, catastrophic-regression prevention, Pareto superiority or universal safety claim follows from the prototype. Valid citations in model explanations establish provenance, not faithful or correct reasoning. These limitations define concrete measurements and engineering prerequisites for the next evaluation.

Ethics Statement

The measurements run local software workloads and involve no human participants or personal data. The REST runtime performs no kernel writes; the experimental adapter changes only its owned child thread’s timer slack and affinity, restores both settings, and performs no host-wide sysctl writes. Broader interventions require an isolated testbed, documented rollback limits and appropriate authorization. Any future human study requires prospective institutional review and informed consent.

AI coding assistants were used in developing parts of the code, prose and analysis. Every reported number is produced by released scripts from raw logs whose SHA-256 hashes are recorded, so that it can be recomputed independently; no generated value substitutes for an unavailable observation.

References

Marah Abdin, Jyoti Aneja, Harkirat Behl, et al. 2024. Phi-4 technical report. arXiv:2412.08905. <https://arxiv.org/abs/2412.08905>.

Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal risk control. In *International Conference on Learning Representations*. <https://arxiv.org/abs/2208.02814>.

Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256.

Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. 2023. Conformal prediction beyond exchangeability. *The Annals of Statistics*, 51(2):816–845. <https://doi.org/10.1214/23-AOS2276>.

Albert Bifet and Ricard Gavaldà. 2007. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, pages 443–448. <https://dblp.org/rec/conf/sdm/BifetG07.html>.

Huilai Chen, Yuanbo Wen, Limin Cheng, Shouxu Kuang, et al. 2024. AutoOS: Make your OS more powerful by exploiting large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 7511–7525. <https://proceedings.mlr.press/v235/chen24at.html>.

Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Mądry. 2024. ContextCite: Attributing model generation to context. In *Advances in Neural Information Processing Systems*, volume 37. <https://arxiv.org/abs/2409.00729>.

DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the frontier of open large language models. arXiv:2512.02556. <https://arxiv.org/abs/2512.02556>.

António Farinhas, Chrysoula Zerva, Dennis Ulmer, and André F. T. Martins. 2024. Non-exchangeable conformal risk control. In *International Conference on Learning Representations*. <https://arxiv.org/abs/2310.01262>.

Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. <https://arxiv.org/abs/2305.14627>.

Yonatan Geifman and Ran El-Yaniv. 2017. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, pages 4878–4887. <https://papers.nips.cc/paper/7073-selective-classification-for-deep-neural-networks>.

Victor Giannakouris and Immanuel Trummer. 2025. λ-Tune: Harnessing large language models for automated database system tuning. *Proceedings of the ACM on Management of Data*, 3(1). <https://doi.org/10.1145/3709652>.

Isaac Gibbs and Emmanuel Candès. 2021. Adaptive conformal inference under distribution shift. In *Advances in Neural Information Processing Systems*, volume 34. <https://proceedings.neurips.cc/paper/2021/hash/0d441de75945e5acbc865406fc9a2559-Abstract.html>.

Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, Greg Kochanski, John Karro, and D. Sculley. 2017. Google Vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1487–1495. <https://research.google/pubs/pub46180/>.

684	Google DeepMind. 2026a. Gemini 3.1 Pro model card. Model card. https://deepmind.google/models/model-cards/gemini-3-1-pro/ .	s.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html .	740 741
687	Google DeepMind. 2026b. Gemini 3.8 Flash model card. Model card. https://deepmind.google/models/model-cards/gemini-3-8-flash/ .	Hongyu Lin, Yuchen Li, Haoran Luo, et al. 2025. TuneAgent (first released as OS-R1): Agentic operating system kernel tuning with reinforcement learning. arXiv:2508.12551. https://arxiv.org/abs/2508.12551 .	742 743 744 745 746
690	Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 herd of models. arXiv:2407.21783. https://arxiv.org/abs/2407.21783 .	Linux man-pages project. 2026a. PR_SET_TIMERSLACK(2const). https://man7.org/linux/man-pages/man2/PR_SET_TIMERSLACK.2const.html .	747 748 749 750
694	Herodotos Herodotou, Yuxing Chen, and Jiaheng Lu. 2020. A survey on automatic parameter tuning for big data processing systems. <i>ACM Computing Surveys</i> , 53(2):43:1–43:37. https://doi.org/10.1145/3381027 .	Linux man-pages project. 2026b. sched_setaffinity(2). https://man7.org/linux/man-pages/man2/sched_setaffinity.2.html .	751 752 753
699	Alon Jacovi and Yoav Goldberg. 2020. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In <i>Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics</i> , pages 4198–4205. https://aclanthology.org/2020.acl-main.386/ .	Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In <i>Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing</i> . https://arxiv.org/abs/2305.14251 .	754 755 756 757 758 759 760 761
705	Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, et al. 2022. Language models (mostly) know what they know. arXiv:2207.05221. https://arxiv.org/abs/2207.05221 .	NVIDIA. 2026. Nemotron 3 Ultra: Open, efficient mixture-of-experts hybrid Mamba-transformer model for agentic reasoning. Technical report. https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Ultra-Technical-Report.pdf .	762 763 764 765 766 767
711	Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In <i>International Conference on Learning Representations</i> . https://arxiv.org/abs/2302.09664 .	Victor Quach, Adam Fisch, Tal Schuster, Adam Yala, Jae Ho Sohn, Tommi S. Jaakkola, and Regina Barzilay. 2024. Conformal language modeling. In <i>International Conference on Learning Representations</i> . https://arxiv.org/abs/2306.10193 .	768 769 770 771 772
716	Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, et al. 2023. Measuring faithfulness in chain-of-thought reasoning. arXiv:2307.13702. https://arxiv.org/abs/2307.13702 .	Qwen Team. 2024. Qwen2.5 technical report. arXiv:2412.15115. https://arxiv.org/abs/2412.15115 .	773 774 775
722	Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A manual-reading database tuning system via GPT-guided Bayesian optimization. <i>Proceedings of the VLDB Endowment</i> , 17(8):1939–1952. https://doi.org/10.14778/3659437.3659449 .	Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. 2023. Measuring attribution in natural language generation models. <i>Computational Linguistics</i> , 49(4). https://arxiv.org/abs/2112.12870 .	776 777 778 779 780 781
729	Hyeonmyeong Lee, Sungmin Jung, and Heeseung Jo. 2022. STUN: Reinforcement-learning-based optimization of kernel scheduler parameters for static workload performance. <i>Applied Sciences</i> , 12(14):7072. https://doi.org/10.3390/app12147072 .	Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. 2012. Practical Bayesian optimization of machine learning algorithms. In <i>Advances in Neural Information Processing Systems</i> , volume 25, pages 2951–2959. https://proceedings.neurips.cc/paper/2012/hash/05311655a15b75fab86956663e1819cd-Abstract.html .	782 783 784 785 786 787 788
735	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In <i>Advances in Neural Information Processing Systems</i> , volume 33, pages 9459–9474. https://proceedings.neurip	Immanuel Trummer. 2022. DB-BERT: A database tuning tool that “reads the manual”. In <i>Proceedings of the 2022 International Conference on Management of Data (SIGMOD)</i> . https://doi.org/10.1145/3514221.3517843 .	789 790 791 792 793

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2305.04388>.

Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic database management system tuning through large-scale machine learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, pages 1009–1024. <https://doi.org/10.1145/3035918.3064029>.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*. <https://arxiv.org/abs/2203.11171>.

An Yang et al. 2025. Qwen3 technical report. arXiv:2505.09388. <https://arxiv.org/abs/2505.09388>.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*. <https://arxiv.org/abs/2309.03409>.

Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, et al. 2019. An end-to-end automatic cloud database tuning system using deep reinforcement learning. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pages 415–432. <https://doi.org/10.1145/3299869.3300085>.

A SemantOS Testbed Details

Design hypotheses. SemantOS is motivated by three hypotheses. First, a configuration’s utility may depend on workload and hardware. Second, joint knob effects may be non-additive. Third, performance, failure probability and intervention cost may conflict. Each hypothesis requires measured interventions with explicit baselines.

For a loss L , define an interaction contrast from four independently measured configurations:

$$I_{ab} = L(a, b) - L(a, b_0) - L(a_0, b) + L(a_0, b_0).$$

For lower-is-better loss, negative I_{ab} indicates a beneficial non-additive interaction relative to the chosen baseline. A bootstrap interval must resample independent runs, not individual requests from a single run. The sign can change with workload, value range or kernel version, so an edge needs its scope, configuration values, measurement IDs and

Algorithm 1 Implemented dry-run bundle path

Require: Validated bundle B , fixed score function, calibration threshold τ

- 1: Validate every member and joint constraints before changing state
- 2: **if** another bundle is active **then**
- 3: Return conflict
- 4: **end if**
- 5: **if** $\max_{a \in B} u(a) \geq \tau$ **then**
- 6: Record veto for all members; return
- 7: **end if**
- 8: Stage the complete bundle in memory at 5%
- 9: **for** requested stage in $\{25, 50, 100\}\%$ **do**
- 10: Advance the simulated bundle; record the transition
- 11: **end for**
- 12: On cancellation, clear the complete simulated bundle
- 13: Never infer an unsafe/safe training label from simulated completion

uncertainty interval. Such an edge describes an empirical interaction, not an unconditional causal coupling.

A DEPENDS-ON edge requires a validated admissibility rule or measured range conditioning; it cannot be inferred solely from the sign of I_{ab} . The released seed graph is empty until edges pass this test. The principal question is whether a reasoner can use independently acquired evidence more effectively than an optimizer with the same inputs, after charging the cost of acquiring that evidence.

Bundle contract. A request contains a nonempty bundle identifier and a list of recommendations with id, knob, proposed, uncertainty, and optional rationale and provenance. Each request submits one bundle. Duplicate knobs, nonfinite scores, unsupported controls, and example-policy bound violations reject the entire request. Dirty-ratio changes must supply both values and satisfy the example policy that the background ratio is less than the foreground ratio. This is a declared policy, not proof that a configuration improves any workload. If any score satisfies $u \geq \tau$, all members are vetoed; otherwise all are staged together. A concurrent bundle receives a conflict response. Dry-run results return `applied=[]` and list simulated members separately.

Deployment requirements. A real actuator must snapshot all old values, validate the full target con-

figuration and available host interfaces, serialize updates, verify readback, and restore every old value on failure. Multiple sysctl writes are not a kernel transaction: partial exposure must be measured and cannot be described as atomic hardware state. Canary shares require separate hosts or an appropriate isolation mechanism; changing a host-global sysctl cannot isolate a fraction of requests on that host. Missing, stale or malformed end-to-end telemetry must stop promotion. These capabilities are requirements, not features demonstrated by the dry-run state machine.

One illustrative decision trace. Consider supplied context with example values 5 and 15 for the two dirty ratios, and retrieved evidence IDs e_1, t_1 . Suppose all three model samples agree on this pair and $w = 0$, giving $u = 0.30$. If an independently obtained threshold is $\tau = 0.25$, the whole bundle is vetoed; if $\tau = 0.35$, it can enter the dry-run canary after joint validation. The audit record includes the values, scores, threshold and decision. These numbers illustrate the contract and are not a measured recommendation or calibrated operating point. The initial artifact threshold is zero until calibration data is explicitly supplied.

Expected cost of gated acceptance. Let $Y = 1$ denote an independently defined unsafe outcome and $A = 1$ acceptance. An outcome label is not defined by whether a rollback happened. For example, a benchmark can predeclare a request deadline, count observed failures or deadline exceedances, and define an action-level breach from these raw observations in a fixed window. Rejected actions have unknown counterfactual outcomes unless evaluated in an isolated experiment; they must not automatically be labeled safe or unsafe.

Theorem 1 (Cost identity). *For costs $c_{FN}, c_{FP}, \lambda \geq 0$, define $C = c_{FN}\mathbf{1}\{Y = 1, A = 1\} + c_{FP}\mathbf{1}\{Y = 0, A = 0\} + \lambda D$, where $D \geq 0$ is measured exposure or SLO debt. Write $\pi = \Pr(Y = 1)$, $q = \Pr(A = 1 \mid Y = 1)$ and $r = \Pr(A = 0 \mid Y = 0)$. Then*

$$\mathbb{E}C = c_{FN}\pi q + c_{FP}(1 - \pi)r + \lambda \mathbb{E}D.$$

For a zero-probability class its associated product is zero.

Proof. Expand each indicator expectation as its joint probability and use the definition of conditional probability. Linearity requires no independence. $\square$

This is an accounting identity, not a claim of novelty or a safety bound. A bound $q \leq \alpha$ controls misses among unsafe actions. A different bound $\Pr(Y = 1 \mid A = 1) \leq \alpha$ instead gives $\Pr(Y = 1, A = 1) \leq \alpha \Pr(A = 1)$, not $\pi\alpha$. Likewise r is a veto rate among safe actions, not the proportion of vetoed actions that are safe. We report all denominators separately.

For bundle-level calibration, u denotes the maximum member score and Y the independently observed bundle outcome. Per-knob labels alone do not establish a bundle-level guarantee.

Selecting a smaller threshold by a cost rule cannot enlarge the acceptance event. Selecting a larger threshold can invalidate the bound; therefore the implementation uses the minimum of the rank threshold and optional cost-selected cap. No unsafe calibration examples implies veto-all. Calibration-window diagnostics are not test-set estimates.

Drift and selective observation. Sliding windows and a drift detector (Bifet and Gavalda, 2007) do not guarantee exchangeability or a deterministic recovery time. Under an explicitly justified total-variation bound on the *joint calibration–test law*, the probability of the same event changes by at most that bound; a marginal score-distance statement alone does not establish it. We neither estimate that distance nor claim recovery. Online labels selected by acceptance introduce additional selection bias. The runtime holds its threshold at zero on detected change until an explicit refresh; refresh remains an exploratory operation, not proof of safety.

Protocol requirements beyond this study. Workload, hardware and session groups are fixed before graph induction and assigned to disjoint train, retrieval, calibration and test roles; hashes of graphs, sources, prompts and calibrators are frozen before test results are opened, and a validator rejects overlap or unknown lineage. For each bundle we count proposed, rejected, vetoed, accepted, staged, completed and rolled-back outcomes, report gate TP/FP/FN/TN with both $\#unsafe-accepted/\#unsafe$ and $/\#accepted$, and leave unknown labels unknown; rollback precision needs independently labeled executed actions. A lower threshold accepts a subset of fixed candidates, but the unsafe fraction among survivors need not be monotone. Absolute per-workload metrics precede ratios against a named baseline; a Pareto claim needs several measured operating points per

Table 5: Follow-up selector replay: mean of ten test-run P95 values. Misses are observed lateness above 200 μ s. Shared rows across selectors are intentional: selectors choosing the same configuration use the same held-out observations.

Period	Selector	P95 (μ s)	Misses
5	control	179.7	13/400
5	empirical min	179.7	13/400
5	graph min	179.7	13/400
5	seeded random	811.6	248/400
8	control	155.7	13/400
8	empirical min	155.7	13/400
8	graph min	155.7	13/400
8	seeded random	642.2	225/400

method. Drift studies must keep the threshold trajectory and held-out outcomes at each decision, and faithfulness is tested by deleting cited and matched uncited context under fixed seeds.

B Additional Measurements

B.1 Selector Replay on the Follow-up

The empirical selector minimizes mean training-run P95 over the four configurations. A saturated two-factor representation encodes the same four means as a baseline, two main effects and their interaction; minimizing it adds no information to table lookup. Both selectors have the same 40 training runs and zero online search trials. An unchanged-control selector and a seeded uniform-random selector provide references. Choices are frozen before calibration and test. We evaluate these fixed selectors by paired *offline replay* on the same measured test grid, not by separate closed-loop executions. A block is the uncertainty unit, and paired reductions are calculated within each period against its explicit control.

Both informed selectors chose 50 μ s slack and all CPUs, identical to control; their paired improvement is exactly zero in this replay. Control P95 is 179.7 ± 102.0 and $155.7 \pm 64.4 \mu$ s at 5 and 8 ms (95% Student-*t* half-width across ten blocks). The training interaction contrast is -18.6μ s; the pre-specified normalized-contrast bootstrap emitted no edge because its interval included zero. We do not turn a nonsignificant contrast into a dependency rule or a semantic-reasoning gain. Acquisition took 6.54 s for training, 8.29 s for retrieval and 10.15 s for calibration; graph construction and selection took 0.10 s, included in calibration-phase wall time.

Table 6: Measured local baseline. Mean of run medians, and mean of run P95 values with 95% Student-*t* CI half-width across 10 runs. Last column is the percentage of errors or operations exceeding a predeclared 50 ms deadline. No tuning gain is implied.

Workload	Median ms	P95 ms	Events %
file_fsync	1.903	2.035 ± 0.043	0.00
sqlite_commit	1.870	2.187 ± 0.273	0.00
zlib_compress	1.474	2.038 ± 0.356	0.00

B.2 Staged Actuation

Across 80 fresh executions, 66 stop on a stage breach and 14 complete all stages; all 80 restore and read back both original values. At 5 and 8 ms, observed deadline misses are respectively 186/816 and 190/880; summed exposure durations are 4.106 and 7.066 s. Early stops reduce the number of observed events, so these fractions cannot be compared directly with fixed-duration, no-stop runs to claim an anomaly reduction. The stop criterion itself is not independent ground truth for rollback precision.

B.3 Local Baseline Measurements

We executed three bounded microbenchmarks on the available host: an Intel Core i5-3570 with four logical CPUs, approximately 31.2 GiB RAM, and Linux 6.17.0-23-generic. These runs characterize the measurement pipeline: no kernel knob was modified and no LLM or controller was compared.

Each workload had 10 separate invocations of 100 measured operations, preceded by 10 warm-up operations. Workload-run order was shuffled with seed 4088. SQLite inserts a 4 KiB value and commits each transaction in WAL mode with synchronous=FULL. The file test overwrites a 64 KiB region and calls fsync. The compression test compresses a fixed seeded 64 KiB payload at zlib level 6 and verifies decompression. Latency is wall-clock operation duration from a monotonic nanosecond clock; the compression measurement includes verification. The raw log retains every operation, run ID, deadline and error flag, with a SHA-256 hash and software/environment manifest.

The anomaly denominator is all measured operations, including failures; an operation counts once if either it fails or exceeds the deadline. This label is independent of veto and rollback. Zero events in these short runs is not evidence of zero population risk. We do not pool heterogeneous workload latencies or infer kernel tuning effects from these

Table 7: Held-out kernel intervention measurements. P95 is the mean of ten run-level P95 wake-up latenesses. Misses count lateness exceeding the predeclared 200 μ s deadline. CPU affinity is all four CPUs or one CPU.

Period	Slack/CPU	P95 (μ s)	Misses
2	50us/all	89.5	2/800
2	50us/one	109.4	18/800
2	1ms/all	1016.8	800/800
2	1ms/one	968.9	771/800
5	50us/all	94.2	2/800
5	50us/one	133.8	22/800
5	1ms/all	1022.6	663/800
5	1ms/one	1073.7	680/800

baseline timings.

B.4 Temporal-Holdout Kernel Interventions

We additionally executed real Linux timer-slack and CPU-affinity interventions in fresh owned child processes, without modifying host-wide sysctls. The two factors were timer slack (50 μ s or 1 ms, set by PR_SET_TIMERSLACK) and affinity (all four available CPUs or CPU 0). Every child checked kernel readback before recording events. This isolated measurement driver is separate from the dry-run SemantOS actuator.

For each periodic wake-up interval (2 or 5 ms), five training blocks and ten subsequent test blocks evaluated all four configurations. Within each block, the eight interval/configuration cases were randomly ordered using seed 4088. Each fresh process performed ten warm-up and 80 measured wake-ups against absolute monotonic-clock targets. The plan was written before execution; the configuration minimizing mean training-run P95 lateness was frozen before test execution. This is temporal holdout within one host and workload family, not the independent workload/hardware split required for claims of generalization. The complete 120-process experiment retained 9,600 event timestamps, readbacks, frozen selections and hashes. No external competing load was introduced.

The training selector chose the explicit control (50 μ s slack, all CPUs) at both intervals: there is no selected-policy improvement over that control. Its test P95 was 89.5 ± 9.2 and 94.2 ± 6.9 μ s at 2 and 5 ms, respectively (95% Student-*t* half-width across ten runs). Raising slack to 1 ms with all CPUs increased P95 by 927.3 ± 16.4 and 928.4 ± 10.0 μ s in paired blocks. Affinity-only differences were 19.9 ± 36.7 and 39.6 ± 62.6 μ s; these intervals include zero. The slack effect is consistent with

timer coalescing; no power measurement, graph discovery, LLM advantage or staged rollback is inferred. A miss is defined by observed wake-up time, independently of any safety gate.

C Prompt and Output Schema

Every model receives the following system prompt:

Select one allowed config to minimize periodic wake-up P95 latency. Return JSON with config (exact allowed name), cited_ids (evidence IDs), and explanation. Evidence contains measured training summaries and retrieval runs, not held-out test outcomes. Do not invent evidence.

The user message is a JSON object with the four allowed configuration names and the evidence list for the input variant. Items have the forms

```
{
  "id": "training-table", "kind": "table",
  "values": {
    "s50000_aall": 132.9413, ...
  }
}
{"id": "interaction", "kind": "graph",
 "contrast_us": -18.62, "edges": [],
 "scope": "same measured table; graph is a reparameterization, not extra evidence"}
{"id": "retrieval-3-0-s50000_aall", "kind": "retrieval", "config": "s50000_aall", "period_ns": 3000000, "p95_us": 126.36}
```

The output schema requires config from the allowed list, cited_ids with at most six entries restricted to the supplied IDs (none for the no-evidence variant), and an explanation of at most 400 characters. Local models enforce it as a llama.cpp JSON grammar with greedy decoding; hosted models receive it as a strict JSON schema through OpenRouter, restricted to providers that honor it. Every answer is re-validated, and an invented or deleted citation makes it invalid. The released records contain each prompt, its SHA-256 hash, the raw response and, for hosted models, the routed provider.